

Why your Claude bot doesn't work



AdiiX @adiix_official

6d • 1 tweets



Tweet

Share ▾

Export ▾



A complete guide to the arbitrage bot and how you can build your own with Claude. After 12 million views, thousands of people tried to build the arbitrage bot. Most failed. Here are the 9 mistakes they made and the exact fixes, step by step.

5 The bot is too slow — the window closes before execution

WHAT'S HAPPENING

The bot correctly identifies an opportunity. The edge calculation is right. But by the time the trade executes, Polymarket has already updated its prices. The opportunity closed. The bot bought at a price that no longer reflects an edge, and it's now in a losing or break-even position.

The arbitrage window on Polymarket is approximately 2.7 seconds in 2026. That's the average time between when a price moves on Binance and when Polymarket adjusts. Your bot needs to identify, calculate, and execute within this window. Total round-trip time — from price detection to order confirmation — should be under 800ms. Most people's bots are running at 2–4 seconds because of three common bottlenecks.

THE THREE BOTTLENECKS AND HOW TO FIX THEM

- **Bottleneck 1 — API polling:** Replace `requests.get()` loops with any async calls. Polling blocks the thread. Ask Claude: *"Replace all synchronous API calls with async/await."*
- **Bottleneck 2 — Market data fetching:** Cache market data locally and refresh it every 500ms in the background. Don't fetch fresh market data on every trade evaluation.
- **Bottleneck 3 — RPC latency:** Use a dedicated Polygon RPC endpoint, not a public one. Free public endpoints add 200–500ms of latency. Alchemy or Infura free tiers are fast enough and take 5 minutes to set up.

What happened after 12 million people read part one

The first article got 12 million views. Thousands of people copied the prompt, opened Claude, and started building their own Polymarket arbitrage bot. Then the DMs started coming in.

"My bot generates but never places a trade." "The API keeps throwing 401 errors." "It runs in paper mode fine but crashes when I go live." "I've been running it for three days and the win rate is 34%." "Claude wrote the code but I can't figure out how to actually run it."

Every single one of these problems is fixable. Every single one of them also has a specific cause not bad luck, not a broken platform, not a defective model. A fixable mistake in setup, configuration, or strategy that happens at a predictable point in the process.

This article documents all nine of them. For each one: what the mistake is, why it happens, what it looks like when it's happening to you, and the exact fix.

Before you read further: If you haven't read the original article How Claude turned \$300 into \$2,382,780 start there. This guide assumes you understand how latency arbitrage works and have already attempted to build a bot. If you're starting from zero, part one is the foundation.

https://x.com/adiix_official/status/2037604599398322682

How to build your own from A to Z

Everything that follows is a complete operational guide. If you follow each step in sequence and don't skip the risk management setup, you'll have a running bot by the end of the day. If you skip steps especially step 4 and step 5 you will lose money. That's not a hedge. It's just what happens.

• Set up a Polymarket wallet

Go to [Polymarket](#) and connect MetaMask or Coinbase Wallet. Fund it with USDC via the Polygon network not Ethereum mainnet, not any other chain. Polygon is the network Polymarket runs on, and using the wrong network means your funds sit inaccessible. For testing purposes, \$100–300 is sufficient. For real trading with meaningful compounding, start with \$500–1,000. Important: Polymarket is not available to US residents. If you're in the US, you cannot legally access the platform. Check the legal status in your specific country before depositing anything.

• Generate your API credentials

Go to docs.polymarket.com, navigate to the API section, and generate a CLOB API key by signing a message with your wallet. This proves ownership without requiring a password. You'll also need your wallet's private key to sign individual trade transactions. Store this private key in a local environment variable never hardcode it into your bot script, never commit it to GitHub, never share it with anyone. A leaked private key means a drained wallet. This is the one step where operational security genuinely matters.

The 9 mistakes diagnosed and fixed

These are ordered by how early in the process they occur. Mistakes 1–3 prevent the bot from running at all. Mistakes 4–6 let it run but produce wrong results. Mistakes 7–9 let it run and produce results, but destroy the account over time.

SETUP ERROR

1 The private key is wrong — or stored wrong

WHAT'S HAPPENING

The most common failure point. You run the bot, it immediately throws a `401 Unauthorized` or `SignatureError`. Nothing works. The Polymarket API rejects every request.

Why: Polymarket uses two separate credentials — your **API key** and your **wallet private key**. The API key authenticates your account. The private key signs every individual transaction. Most people either mix these up, or they copy the private key incorrectly (wrong format, extra characters, missing the `0x` prefix).

The second issue: storing the private key in the code itself. Claude will often hardcode a placeholder like `PRIVATE_KEY = "your_key_here"` in the generated script. People replace it with their actual key — inside the file. This works but is a serious security risk, and if the key format has any issue, the entire auth chain fails silently.

THE FIX

Store your private key in a `.env` file, never in the script. Create a file called `.env` in the same folder as your bot with this format:

```
• • • .env file – store credentials here, never in the script

POLY_API_KEY=your_api_key_from_polymarket_dashboard
POLY_PRIVATE_KEY=0x_your_wallet_private_key_with_0x_prefix
POLY_CHAIN_ID=137 # Polygon mainnet

# In your bot script, load with:
from dotenv import load_dotenv
load_dotenv()
api_key = os.getenv('POLY_API_KEY')
private_key = os.getenv('POLY_PRIVATE_KEY')

# Add .env to .gitignore immediately. Never commit it.
```

ENVIRONMENT ERROR

2 Dependencies aren't installed — or the wrong version is installed

WHAT'S HAPPENING

The bot throws import errors immediately on startup. `ModuleNotFoundError: No module named 'py_clob_client'`. Or it starts but behaves unexpectedly because an older version of a library has different method signatures than what Claude generated code for.

Claude generates code for current library versions — specifically `py-clob-client` 0.9+. If you install without specifying versions, pip sometimes resolves to an older release with incompatible method names.

THE FIX — EXACT COMMANDS

Run these in order. Don't skip the virtual environment step.

1. Create and activate a virtual environment

```
python3 -m venv bot-env
```

```
source bot-env/bin/activate # Windows: bot-env\Scripts\activate
```

2. Install exact required versions

```
pip install py-clob-client==0.9.0
```

```
pip install websockets==12.0
```

```
pip install python-dotenv==1.0.0
```

```
pip install aiohttp==3.9.0
```

3. Verify the install worked

```
python3 -c "from py_clob_client.client import ClobClient; print('OK')"
```

Expected output: OK

If you see an error, paste it back into Claude with:

"I got this error installing py-clob-client. Fix it."



The Binance WebSocket disconnects and the bot doesn't reconnect

WHAT'S HAPPENING

The bot starts fine. It runs for a few minutes or hours. Then it silently stops making trades. You check the logs and see connection errors. The Binance WebSocket dropped — as they do regularly — and the bot has no reconnection logic. It's now running with stale or no price data, which means it either stops trading entirely or, worse, trades on outdated numbers.

This is the mistake that most commonly affects bots that *seem* to work. The bot runs, it shows activity, but it's actually frozen on the last price it received before the connection dropped. Every "trade" it evaluates is based on a price from 20 minutes ago.

THE FIX — ADD RECONNECTION LOGIC

Paste this prompt into Claude immediately after your bot is generated. This adds robust reconnection handling to the existing code.

Add reconnection logic to the Binance WebSocket handler.

When the connection drops, the bot should:

1. Log the disconnection with a timestamp
2. Wait 5 seconds, then attempt to reconnect (*exponential backoff*)
3. Retry up to 10 times before sending a Telegram alert
4. **Pause all trading** while the connection is down – *do not trade on stale price data*
5. Resume trading automatically once reconnected
6. Add a `last_price_timestamp` check: if the price data is older than 10 seconds, treat it as stale and pause trading

Keep all existing logic. Only modify the WebSocket handler.

4 The edge threshold is set too low — the bot trades noise

WHAT'S HAPPENING

The bot is active. It's placing trades. But the win rate is hovering around 50–60% — essentially random. You're not catching real arbitrage opportunities. You're catching normal price fluctuation and treating it as an edge.

The original prompt in part one specified a **3% minimum edge threshold** and a **5% minimum edge before execution**. Many people modified this — consciously or by accident — to something like 0.5% or 1%. At that level, the difference between Polymarket's price and the Binance price is indistinguishable from normal market noise. You're not arbitraging. You're gambling.

The threshold also needs to account for fees. Each Polymarket trade costs approximately 0.5–1% in fees and price impact. An edge of 1% after fees is essentially zero. You need real edge — not theoretical edge.

THE FIX — CORRECT THRESHOLDS

These are the thresholds that produce consistent positive results based on the 0x8dxd strategy. Do not lower them, even if it means fewer trades.

- **Minimum detected edge:** 5% (do not look at opportunities below this)
- **Minimum execution edge:** 8% (only place trades above this)
- **Confidence threshold:** 85% (the direction must be clear)
- **Position size:** maximum 8% of portfolio per trade
- **Minimum market liquidity:** \$50,000 (ignore smaller markets)

The bot is too slow — the window closes before execution

WHAT'S HAPPENING

The bot correctly identifies an opportunity. The edge calculation is right. But by the time the trade executes, Polymarket has already updated its prices. The opportunity closed. The bot bought at a price that no longer reflects an edge, and it's now in a losing or break-even position.

The arbitrage window on Polymarket is approximately 2.7 seconds in 2026. That's the average time between when a price moves on Binance and when Polymarket adjusts. Your bot needs to identify, calculate, and execute within this window. Total round-trip time — from price detection to order confirmation — should be under 800ms. Most people's bots are running at 2–4 seconds because of three common bottlenecks.

THE THREE BOTTLENECKS AND HOW TO FIX THEM

- **Bottleneck 1 — API polling:** Replace `requests.get()` loops with any async calls. Polling blocks the thread. Ask Claude: *"Replace all synchronous API calls with async/await."*
- **Bottleneck 2 — Market data fetching:** Cache market data locally and refresh it every 500ms in the background. Don't fetch fresh market data on every trade evaluation.
- **Bottleneck 3 — RPC latency:** Use a dedicated Polygon RPC endpoint, not a public one. Free public endpoints add 200–500ms of latency. Alchemy or Infura free tiers are fast enough and take 5 minutes to set up.

The bot is too slow — the window closes before execution

WHAT'S HAPPENING

The bot correctly identifies an opportunity. The edge calculation is right. But by the time the trade executes, Polymarket has already updated its prices. The opportunity closed. The bot bought at a price that no longer reflects an edge, and it's now in a losing or break-even position.

The arbitrage window on Polymarket is approximately 2.7 seconds in 2026. That's the average time between when a price moves on Binance and when Polymarket adjusts. Your bot needs to identify, calculate, and execute within this window. Total round-trip time — from price detection to order confirmation — should be under 800ms. Most people's bots are running at 2–4 seconds because of three common bottlenecks.

THE THREE BOTTLENECKS AND HOW TO FIX THEM

- **Bottleneck 1 — API polling:** Replace `requests.get()` loops with any async calls. Polling blocks the thread. Ask Claude: *"Replace all synchronous API calls with async/await."*
- **Bottleneck 2 — Market data fetching:** Cache market data locally and refresh it every 500ms in the background. Don't fetch fresh market data on every trade evaluation.
- **Bottleneck 3 — RPC latency:** Use a dedicated Polygon RPC endpoint, not a public one. Free public endpoints add 200–500ms of latency. Alchemy or Infura free tiers are fast enough and take 5 minutes to set up.

The bot tracks the wrong contracts — or too many contracts

WHAT'S HAPPENING

The bot runs without errors but never finds good opportunities. Or it finds opportunities constantly but most of them don't resolve correctly. The issue is contract selection.

The 0x8dxd strategy worked specifically on **5-minute and 15-minute BTC/ETH up/down contracts**. These are short-duration contracts with high liquidity and predictable settlement. Other contract types — elections, sports events, longer-duration crypto contracts — have different dynamics that the latency arbitrage strategy doesn't apply to. The edge comes from the specific timing relationship between CEX prices and Polymarket's short-duration settlement.

The second issue is tracking too many markets simultaneously. Monitoring 50+ markets creates computational overhead that slows down the edge detection for the good markets.

THE FIX — CORRECT MARKET SELECTION

- Only monitor **BTC and ETH** contracts
- Only trade **5-minute and 15-minute** up/down contracts
- Minimum **\$50,000 liquidity** in the market before considering it
- Maximum **20 active markets** monitored simultaneously
- Ignore all non-crypto contracts for latency arbitrage



No kill switch — a bad sequence wipes the account

WHAT'S HAPPENING

This is the most dangerous mistake. The bot has been running, producing positive results, and you've become comfortable with it. Then something changes — a market moves in an unusual way, a bug surfaces in an edge case, or fees spike — and the bot starts losing trades in sequence. Without a kill switch, it keeps trading. A bad hour can erase a week of profits. A catastrophic bug can drain the account entirely.

In the March 2026 head-to-head experiment (Claude bot vs OpenClaw), the OpenClaw system was fully liquidated. The Claude-generated code survived because it had robust drawdown protection. The OpenClaw setup did not.

THE FIX — NON-NEGOTIABLE KILL SWITCH RULES

Paste this into Claude as a separate prompt after your bot is generated. Add these rules before you go live under any circumstances.

Add the following risk management rules to the bot.

These must be enforced **before every trade execution**, not after:

Rule 1 – Daily halt: If daily P&L drops below **-20%** of starting balance, halt all trading for the remainder of the calendar day.

Rule 2 – Drawdown kill switch: If total portfolio drops below **60%** of the highest balance ever recorded, halt all trading permanently and send a Telegram alert requiring manual restart.

Rule 3 – Consecutive loss halt: If the bot loses **5 trades in a row**, pause for 30 minutes and send an alert before resuming.

Rule 4 – Position size hard cap: Never exceed **8%** of current portfolio in a single position. Recalculate this limit before every trade.

These rules override all strategy logic. If in doubt, don't trade.

The bot runs silently — you don't know when it breaks

WHAT'S HAPPENING

The bot is running somewhere — on your computer, on a server. You check it once a day. But you don't know when it stopped working, when it hit an error, when the WebSocket dropped and never recovered. You find out three days later when you notice the trade count hasn't moved.

Silent failures are the most costly kind. The bot isn't losing money — but it isn't making money either, and you're paying for compute and subscriptions while it sits idle.

THE FIX — MINIMUM MONITORING SETUP

- **Telegram bot alerts:** Every trade entry and exit. Every error. Every kill switch trigger. This takes 10 minutes to set up and is the single most important observability tool.
- **Heartbeat message:** Every hour, the bot sends a Telegram message: current balance, trades today, win rate today. If you don't get this message, the bot is down.
- **Log file with rotation:** All activity logged to a file with timestamps. Cap at 50MB and rotate. This is your audit trail.
- **Run on a VPS, not your laptop:** A bot running on your personal computer stops when the screen sleeps, when you close the lid, when you restart. A \$5/month VPS runs 24/7.

Going live before paper trading proves it works

WHAT'S HAPPENING

You got excited. The bot ran for a few hours in paper mode, looked good, and you switched it to live trading with \$500. Within days, it underperformed the paper simulation significantly — or lost money outright. The paper results didn't translate.

Paper trading and live trading differ in three important ways that most people don't account for:

- **Slippage:** In paper mode, you always "get" the price you saw. In live trading, by the time your order executes, the price has moved. On a 2.7-second window, this is the difference between a 5% edge and a 2% edge.
- **Gas fees:** Every Polygon transaction costs gas. Depending on network congestion, this can be \$0.01 or \$0.20 per trade. At 200 trades/day, that's up to \$40/day in fees that don't appear in paper mode.
- **Market impact:** In paper mode, your size doesn't affect the market. In live trading, if you're buying into a thin market, your own order moves the price against you.

THE FIX — CORRECT PAPER-TO-LIVE TRANSITION

- Run paper mode for a **minimum of 7 days** — not hours
- Require a **paper win rate above 75%** before going live
- Require a **minimum 200 paper trades** — not 20
- Start live trading at **\$1–5 per trade**, not your full position size
- Compare live win rate to paper win rate daily for the first week — if they diverge by more than 10%, stop and investigate

Wrong vs right the full comparison

Here is the complete picture of what a broken setup looks like versus a working one. Use this as a reference checklist before you go live.

BROKEN BOT VS WORKING BOT – SIDE BY SIDE		COMMON PATTERNS
PARAMETER	BROKEN SETUP	WORKING SETUP
Private key storage	Hardcoded in script	In .env file, loaded via dotenv
Min edge threshold	0.5–2% (trades noise)	5% detect, 8% execute
WebSocket handling	No reconnection logic	Auto-reconnect + stale data check
Execution speed	2–4 seconds round-trip	Under 800ms end-to-end
Kill switch	None	–20% daily halt, –40% permanent halt
Contract types	All markets, all types	BTC/ETH 5min + 15min only
Paper testing	Few hours, then live	7+ days, 200+ trades, 75%+ win rate
Monitoring	Manual checks once a day	Telegram alerts + hourly heartbeat
Position sizing	Fixed amount or instinct	Kelly Criterion, max 8% portfolio
Running environment	Personal laptop	VPS (24/7 uptime)

The correct prompt updated and battle-tested

After seeing what broke in thousands of real attempts, here is the updated prompt. It

incorporates all nine fixes from the start so you don't have to patch them in afterwards.

Copy this verbatim. Do not simplify it. Every line is there for a reason.

Build me a production-ready Python latency arbitrage bot

for Polymarket with ALL of the following requirements:

CREDENTIALS:

- Load all keys from a .env file using python-dotenv
- Never hardcode credentials in the script
- Required env vars: POLY_API_KEY, POLY_PRIVATE_KEY, TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, ALCHEMY_RPC_URL

PRICE MONITORING:

- Connect to Binance WebSocket: wss://stream.binance.com:9443
- Track BTC/USDT and ETH/USDT ticker streams
- Auto-reconnect on disconnect with exponential backoff (max 10 retries)
- Reject price data older than 10 seconds as stale
- Pause all trading when price data is stale

EDGE DETECTION:

- Minimum detectable edge: 5% (ignore anything below)
- Minimum execution edge: 8% (only trade above this)
- Only monitor BTC/ETH contracts, 5-minute and 15-minute duration
- Minimum market liquidity: \$50,000
- Maximum 20 markets monitored simultaneously

POSITION SIZING:

- Use half-Kelly Criterion for every position
- Hard cap: maximum 8% of current portfolio per trade
- Recalculate portfolio value before every trade

RISK MANAGEMENT (non-negotiable):

- Daily halt if P&L falls below -20% of day-start balance
- Permanent halt if portfolio falls below 60% of all-time high
- 30-minute pause after 5 consecutive losses
- All halts send Telegram alert and require manual restart to resume

EXECUTION:

- Full async architecture (asyncio + aiohttp, no blocking calls)
- Use Alchemy RPC from env var for Polygon transactions
- Total execution time target: under 800ms from detection to order

MONITORING:

- Telegram alert on every: trade entry, trade exit, error, kill switch
- Hourly heartbeat: balance, today's trades, today's win rate
- Rotating log file: all activity with timestamps, 50MB max

PAPER MODE:

- Paper mode enabled by default
- Three explicit flags required to enable live: `--live --confirm --i-understand-risks`
- Paper mode logs all simulated trades to SQLite

Use Python 3.11+. Include a `requirements.txt` with pinned versions.

Start in paper mode. Do not enable live trading without explicit flags.

After Claude generates the code: Do not run it immediately. First check that it created a `requirements.txt` file. If not, ask Claude: *"Generate a requirements.txt with all dependencies pinned to specific versions."* Then create your `.env` file, install dependencies, and run in paper mode for 7 days minimum.

The correct setup step by step

This is the sequence that produces a working bot. Follow it in order. Do not skip steps, especially 4 and 5.

Generate the bot with the updated prompt

Open Claude.ai or Claude Code. Paste the full prompt from section 04 above verbatim, nothing removed. Claude will generate a complete Python codebase. It should include a main bot file, a `requirements.txt`, and a `.env.example` template.

If any of these are missing, ask Claude: "You didn't generate [requirements.txt / .env.example]. Add it." Don't proceed without all three files.

Set up the environment

Create a project folder. Save the bot files there. Create a virtual environment with `python3 -m venv bot-env` and activate it. Install dependencies with `pip install -r requirements.txt`.

Copy `.env.example` to `.env` and fill in your credentials. Add `.env` to `.gitignore` before doing anything else.

Get your credentials

Polymarket API key: Go to docs.polymarket.com → API Keys → generate via your wallet

signature. This authenticates your account.

Wallet private key: Export from MetaMask (Settings → Security → Export Private Key). This signs transactions. It must start with 0x.

Alchemy RPC: Create a free account at alchemy.com → Create App → Polygon Mainnet → copy the HTTPS endpoint. This replaces slow public RPC nodes.

Telegram bot: Message @BotFather on Telegram → /newbot → copy the token. Then message your bot and get your chat ID from the API.

Run paper mode for 7 days minimum

Start the bot with `python3 bot.py` paper mode is the default. Do not modify this.

Watch the first 20–30 trades manually. Verify: the edge calculations look right, the bot is not trading on stale data, the Telegram alerts arrive correctly, the logs are writing.

Your paper trading must hit all three before you go live:

- At least 200 completed paper trades
- Paper win rate above 75%
- At least 7 consecutive days of running without crashes

If paper mode doesn't hit these numbers, the bot has a problem. Find it before you trade real money.

Deploy to a VPS before going live

A \$5/month VPS on DigitalOcean, Linode, or Vultr gives you 24/7 uptime. Running the bot on your laptop means it stops when you sleep, travel, or restart.

Transfer your bot files (not your `.env` create a new one on the server). Install dependencies. Run paper mode on the VPS for 48 hours to confirm it's stable there too.

Go live small, then scale

When paper mode passes all three thresholds and the VPS is stable, start live trading with the flags: `python3 bot.py --live --confirm --i-understand-risks`

First week of live trading: watch every single trade. Compare live win rate to paper win rate daily. If they diverge by more than 10%, stop and investigate before continuing.

Scale position sizes gradually only when you have 50+ live trades of data confirming the system works in real conditions.

The debug checklist if it's still broken

If you've followed everything above and the bot still isn't working, go through this checklist in order. Every item here was a real failure point reported by readers of part one.

- ☐ Private key starts with 0x and has no extra spaces or characters
- ☐ API key and private key are in a .env file, not hardcoded in the script
- ☐ .env file is in the same directory as the bot script
- ☐ Virtual environment is activated before running the bot
- ☐ All dependencies installed via requirements.txt inside the virtual environment
- ☐ Minimum edge threshold is set to 5% detect, 8% execute — not lower
- ☐ Bot only monitors BTC/ETH 5-minute and 15-minute contracts
- ☐ WebSocket reconnection logic is present and tested
- ☐ Stale price data check is active (reject prices older than 10 seconds)
- ☐ Kill switch configured: -20% daily halt, -40% all-time-high halt